ASSOCIATION RULE MINING

Apriori Algorithm

1. Why Should You Care? Real-World Examples

Association rule mining is just a formal way of answering:

> *"When people buy / do / have **X**, what else do they usually buy / do / have?"*

#	Example	Pattern Found	Business Action
1	Walmart's beer & diapers	Men buying diapers often also bought beer	Placed products near each other and bundled discounts
2	Bread & jam	Customers buying bread often bought jam too	Shelved them next to each other
3	Potatoes & onions	Common combination in everyday cooking	Bundled together with a discount
4	Medical symptoms	Certain symptoms tend to occur together (co-morbidity)	Helps doctors diagnose and treat more accurately

The general idea: if items A and B are frequently found together, you can:

- Place them near each other (physically or on a webpage)
- Discount one to drive sales of both
- Advertise A to people who already buy B
- Even design a new product that combines A and B

This is exactly what association rule mining helps you discover automatically from data — instead of relying on a salesperson noticing a pattern by chance.

2. What Is Association Rule Mining?

Definition: A data mining technique that finds relationships between items that frequently co-occur in a dataset (most commonly, a database of transactions), and expresses them as "if this, then that" rules.

Formal Notation

- $I = \{i_1, i_2, i_3, ..., i_n\}$ → the set of all possible items
- $D = \{t_1, t_2, ..., t_n\}$ → the database of transactions (each transaction is a subset of I)

An association rule is written:

$$X \rightarrow Y$$

Where:

- $X \subseteq I$ and $Y \subseteq I$ (both are itemsets)
- $X \cap Y = \emptyset$ (they share no items)
- X = Antecedent (the "IF" part)
- Y = Consequent (the "THEN" part)

Illustrative Example

Transaction	Items
T1	Alpha, Beta, Gamma
T2	Alpha, Beta, Epsilon
T3	Alpha, Beta, Theta
T4	Beta, Gamma
T5	Alpha, Epsilon

Item counts: **Alpha** = 4 (T1, T2, T3, T5), **Beta** = 4 (T1, T2, T3, T4).

Transactions containing **both Alpha and Beta**: T1, T2, T3 → count = 3.

> *Confidence (Alpha → Beta) = 3 / 4 = 75%*

This means: 75% of the transactions that contain Alpha also contain Beta — a strong (but not perfect) rule.

3. The Three Key Metrics: Support, Confidence & Lift

These three numbers are the heart of association rule mining. Learn them well.

3.1 Support — "How popular is this combination?"

$$Support(X) = \sigma(X) / N$$

- σ(X) = number of transactions containing itemset X
- N = total number of transactions

Example: If "Bread" appears in 4 out of 5 transactions:

$$Support\ (Bread) = 4/5 = 0.80 = 80\%$$

3.2 Confidence — "If X happens, how likely is Y?"

$$Confidence(X \rightarrow Y) = \sigma(X \cup Y) / \sigma(X)$$

Example: If {Bread, Jam} occurs in 3 transactions and {Bread} occurs in 4:

$$Confidence\ (Bread \rightarrow Jam) = 3/4 = 0.75 = 75\%$$

> *Drawback: Confidence only looks at how popular X is, not Y. If Y happens to be popular on its own (regardless of X), confidence can look artificially high even when there's no real relationship. That's where lift comes in.*

3.3 Lift — "Is the relationship actually meaningful?"

$$Lift(X \rightarrow Y) = Support(X \cup Y) \,/\, (Support(X) \times Support(Y))$$

Lift Value	Meaning	Interpretation
> 1	Positive association	Buying X makes buying Y more likely ✅
= 1	No association	X and Y are statistically independent ⭕
< 1	Negative association	Buying X makes buying Y less likely ❌

3.4 Side-by-Side Comparison

Metric	Formula	What It Tells You
Support	$\sigma(X) / N$	How common the itemset is overall
Confidence	$\sigma(X \cup Y) / \sigma(X)$	Reliability of the "if X then Y" rule
Lift	$Support(X \cup Y) / (Support(X) \times Support(Y))$	Whether the rule is a real pattern or coincidence

4. Frequent Itemsets

Term	Meaning	Example
Itemset	Any set of items grouped together	{Bread, Jam}
k-itemset	An itemset with exactly k items	{Bread, Jam} is a 2-itemset
Frequent itemset	An itemset whose support meets or exceeds the minimum support threshold	Support ≥ chosen cutoff (e.g., 3 transactions)

Two thresholds drive the whole algorithm, and you (the analyst) choose them up front:

- Minimum support — how "popular" must an itemset be to matter?
- Minimum confidence — how "reliable" must a rule be to be reported?

5. The Apriori Algorithm

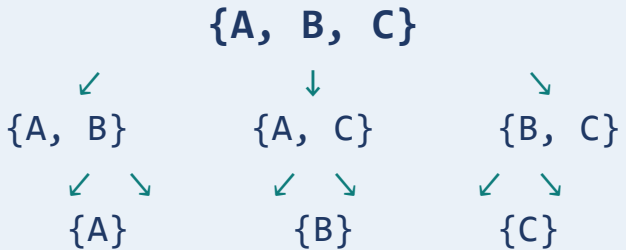
Attribute	Detail
Proposed by	R. Agrawal and R. Srikant
Year	1994
Goal	Efficiently find all frequent itemsets in a transaction database
Name origin	Uses prior knowledge about frequent itemset properties
Strategy	"Bottom-up" — starts with single items and grows itemsets one item at a time

5.1 The Golden Rule: Anti-Monotonicity

This single property is what makes Apriori efficient (instead of having to check every possible combination of items):

> ✓ *If an itemset is frequent, every one of its subsets must also be frequent.*
>
> ✗ *If an itemset is infrequent, every one of its supersets must also be infrequent.*

Visually — a frequent 3-itemset requires every smaller piece beneath it to already be frequent:



This means: the moment a smaller itemset fails the support test, you can immediately throw away every larger itemset built on top of it — without even counting it. That's a huge time-saver.

5.2 The Two Core Steps

Step	What Happens
Join	Combine frequent k-itemsets with each other to build (k+1)-itemset candidates
Prune	Scan the database, count each candidate's actual support, and discard any that fall below the minimum threshold

5.3 Full Algorithm Flow

1. Set k = 1
2. Count every individual item → find frequent 1-itemsets (L_1)
3. k = k + 1
4. Join: build candidate k-itemsets from the frequent (k−1)-itemsets
5. Prune: remove any candidate whose support is below the minimum, or whose subsets aren't all frequent
6. If no frequent itemsets remain at this level → stop and move to rule generation; otherwise, go back to step 3
7. Generate all possible association rules from the final frequent itemsets
8. Keep only the rules that meet the minimum confidence threshold

6. Full Worked Example

6.1 The Data

Transaction ID	Items
T1	I1, I2, I3
T2	I2, I3, I4
T3	I4, I5
T4	I1, I2, I3
T5	I1, I2, I3, I4

Parameter	Value
Total transactions (N)	5
Minimum support	3 transactions (60%)
Minimum confidence	60%

STEP 1 — Frequent 1-Itemsets

Formula used: Support(X) = σ(X) / N

Item	Appears In	Count σ	Support	Meets min. support (3)?
I1	T1, T4, T5	3	60%	✓ Yes
I2	T1, T2, T4, T5	4	80%	✓ Yes
I3	T1, T2, T4, T5	4	80%	✓ Yes
I4	T2, T3, T5	3	60%	✓ Yes
I5	T3	1	20%	✗ No — pruned

L_1 (Frequent 1-itemsets): {I1}, {I2}, {I3}, {I4}

STEP 2 — Frequent 2-Itemsets

Join: Combine every pair of items from L_1. Prune: Count each pair's real support and drop anything below 3.

Itemset	Appears In	Count σ	Support	Status
{I1, I2}	T1, T4, T5	3	60%	✓ Frequent
{I1, I3}	T1, T4, T5	3	60%	✓ Frequent
{I1, I4}	T5	1	20%	✗ Pruned
{I2, I3}	T1, T2, T4, T5	4	80%	✓ Frequent
{I2, I4}	T2, T5	2	40%	✗ Pruned
{I3, I4}	T2, T5	2	40%	✗ Pruned

L_2 (Frequent 2-itemsets): {I1,I2}, {I1,I3}, {I2,I3}

STEP 3 — Frequent 3-Itemsets (Anti-Monotone Property in Action)

Before even counting, check whether all 2-item subsets of each 3-item candidate are in L_2. If even one subset is missing, discard the candidate immediately — no need to scan the database for it.

Candidate	Required 2-item Subsets	All Frequent?	Verdict
{I1, I2, I3}	{I1,I2}, {I1,I3}, {I2,I3}	✅ All present in L_2	Keep — verify by counting
{I1, I2, I4}	{I1,I2}, {I1,I4}, {I2,I4}	❌ {I1,I4}, {I2,I4} missing	Pruned
{I1, I3, I4}	{I1,I3}, {I1,I4}, {I3,I4}	❌ {I1,I4}, {I3,I4} missing	Pruned
{I2, I3, I4}	{I2,I3}, {I2,I4}, {I3,I4}	❌ {I2,I4}, {I3,I4} missing	Pruned

Verify the survivor, {I1, I2, I3}, by actually counting it:

Transaction	Has I1?	Has I2?	Has I3?	Has all three?
T1	✓	✓	✓	✅
T2	✗	✓	✓	❌
T3	✗	✗	✗	❌
T4	✓	✓	✓	✅
T5	✓	✓	✓	✅

Count = 3 → Support = 3/5 = 60% ✅ Frequent

L_3 (Frequent 3-itemsets): {I1, I2, I3}

STEP 4 — Attempt Frequent 4-Itemsets

Only one candidate is even possible: {I1, I2, I3, I4}. Check its 3-item subsets against L_3:

Subset	In L_3?
{I1, I2, I3}	✓ Yes
{I1, I2, I4}	✗ No
{I1, I3, I4}	✗ No
{I2, I3, I4}	✗ No

> *Since not all subsets are frequent, {I1, I2, I3, I4} is pruned. No frequent 4-itemsets exist — the algorithm stops here.*

STEP 5 — Generate Association Rules

Every possible non-empty split of {I1, I2, I3} (support = 0.60) becomes a candidate rule. Using:

$$Confidence(X \rightarrow Y) = Support(X \cup Y) / Support(X)$$

Rule	Calculation	Confidence	Meets 60%?
{I1, I2} → {I3}	0.60 / 0.60	100%	✓
{I1, I3} → {I2}	0.60 / 0.60	100%	✓
{I2, I3} → {I1}	0.60 / 0.80	75%	✓
{I1} → {I2, I3}	0.60 / 0.60	100%	✓
{I2} → {I1, I3}	0.60 / 0.80	75%	✓
{I3} → {I1, I2}	0.60 / 0.80	75%	✓

All six rules clear the 60% confidence bar.

STEP 6 — Add Lift to Double-Check the Rules Are Meaningful

$$Lift(X \rightarrow Y) = Support(X \cup Y) \,/\, (Support(X) \times Support(Y))$$

Rule	Calculation	Lift
{I1, I2} → {I3}	0.60 / (0.60 × 0.80)	1.25
{I1, I3} → {I2}	0.60 / (0.60 × 0.80)	1.25
{I2, I3} → {I1}	0.60 / (0.80 × 0.60)	1.25
{I1} → {I2, I3}	0.60 / (0.60 × 0.80)	1.25
{I2} → {I1, I3}	0.60 / (0.80 × 0.60)	1.25
{I3} → {I1, I2}	0.60 / (0.80 × 0.60)	1.25

Every rule has a lift of 1.25 > 1, confirming these aren't random coincidences — buying items from {I1, I2, I3} genuinely increases the likelihood of buying the rest of that set.

7. Results Summary

Frequent Itemsets Found

Level	Itemsets	Support
L_1	{I1}, {I2}, {I3}, {I4}	60%, 80%, 80%, 60%
L_2	{I1,I2}, {I1,I3}, {I2,I3}	60%, 60%, 80%
L_3	{I1,I2,I3}	60%

Final Strong Association Rules (Confidence ≥ 60%, Lift > 1)

Rule	Confidence	Lift	Strength
{I1, I2} → {I3}	100%	1.25	Very strong
{I1, I3} → {I2}	100%	1.25	Very strong
{I1} → {I2, I3}	100%	1.25	Very strong
{I2, I3} → {I1}	75%	1.25	Strong
{I2} → {I1, I3}	75%	1.25	Strong
{I3} → {I1, I2}	75%	1.25	Strong

8. Quick Reference Cheat Sheet

Formulas

Metric	Formula
Support	$\text{Support}(X) = \sigma(X) / N$
Confidence	$\text{Confidence}(X \rightarrow Y) = \sigma(X \cup Y) / \sigma(X)$
Lift	$\text{Lift}(X \rightarrow Y) = \text{Support}(X \cup Y) / (\text{Support}(X) \times \text{Support}(Y))$

Apriori's Two Golden Rules

1. All subsets of a frequent itemset must be frequent.
2. All supersets of an infrequent itemset must be infrequent.

Lift Interpretation

Lift	Meaning
> 1	Positive association ✓
= 1	No association ○
< 1	Negative association ✗

Apriori Advantages & Limitations

Advantages	Limitations
Simple to understand and implement	Can be slow on very large datasets
Anti-monotone pruning drastically cuts down work	Requires multiple full scans of the database
Works well for market basket–style data	May generate a large number of rules to sift through

Association rules help uncover hidden patterns in data, turning raw transaction logs into smarter shelf placement, better recommendations, and genuinely useful business insight.